Project Title: A simple calculator

Objective: The objective of this project is to design and implement a simple calculator system, as described below.The implementation will incorporate classic Gang of Four (GoF) design patterns, as indicated in the sequel.

Project Description: In this project, students will work in groups of 2 to create a simple calculator system, as described below. The calculator is a console application that processes input strings and outputs the result of computations, among other things. The point is to demonstrate the students understand, apply and implement specific design patterns. Correspondingly, a fully functional implementation that does not conform to the project requirements below, especially design pattern usage, is of no value.

Project Requirements:

- **Functional Requirements**: The application requests the user to input string tokens tokens via the console, representing an expression to evaluate (e.g "1234 + 333"). Once the special character "=" is inputted, the system should output the expression result, and resets the current expression. Anything that is composed entirely of digits is a number, otherwise it is considered an operator (unless it is a special command, see below). For simplicity and for focusing on what is more relevant, you can assume all inputted numbers are zero or positive, and that all operators have the same priority,and that all tokens are separated by spaces.

  The system should also include the following features:

  Memory: The user can recover an earlier expression by typing "Mxxx",where xxx is a index identifying the previous expression, starting at 0 (consecutive expressions are delimited by use of the "=" command). The user can list the stored expressions by using the special command "L".

  Editing: The user can edit the current expression by replacing one operator with a different operator in one of the stored expressions. This is achieved by inputing the special command *R expressionindex originalOp newOp*. For example, "R 0 + *" will replace all + with * in the stored expression with index 0.

  The user can also combine expressions with an operator using the command *C firstExpressionIndex secondExpressionIndex operator*. When evaluating the combined expression, both sub-expressions are evaluated first, (i.e. operators are NOT evaluated left to right in this case). See the example of use.

  Example of use:

```
Input> 1 + 2
Input> * 4 =
Result> 12
Input> L
0: (1+2)*4
Input> 1 + 2 =
Result>3
```

```
Input> L
0: (1+2)*4
1: 1+2
Input>M1
Input>+ 3 =
Result>7
Input> L
0: (1+2)*4
1: 1+2
2: (1+2)+3
Input>R 0 * +
Input>L
0: (1+2)+4
1: 1+2
2:(1+2)+3
Input> C 1 2 * =
Result>21
Input> L
0: (1+2)+4
1: 1+2
2: (1+2)+3
3: ((1+2)+4)*(1+2)
```

- **Design Pattern requirements:**
  - The application should use the **State** pattern to ensure that it correctly processes and reports errors in the inputted expression (e.g "1233 222", or "33 + * 44". Provide a description of your state machine in the commentary.
  - The application should use a **Composite** pattern to represent the expression that is being introduced. The application should always use the composite when working with the expression (computing, printing,etc). String representations of the expression are used or generated on the fly only for input/output operations. The generated string representation of expressions should include parenthesis when necessary to indicate order of execution, as demonstrated in the example.
  - The application should use a **Chain of Responsibility** to evaluate the result of an operator (thereby allowing new operators to be easily added to the system).
  - The application should use the **Builder** pattern to support creation of the composite representing the expression.
  - The application should use the **Flyweight** pattern for representing numbers, thereby avoiding unnecessary memory use.
  - The application should use the **Visitor** pattern to make changes to the composite representing the expression when the user replaces operators with the R command.
  - You can use other design patterns for other purposes, if you find it appropriate to do so.

Deliverables and submission: You should submit your code, no later than 31st of October, in the link provided in moodle.

Defense: The project will be subjected to mandatory discussion/defense during class on the 31st October. There will be registration for time slots allocated for this defense.

You will be required to demonstrate the functionality and present and discuss design patterns use in this project. You might also be required to discuss the implementation of potential changes or evolutions of the system, with respect to design pattern use. You are free to use whatever resources you want to support your development, but you are assumed to have complete ownership of the codebase you submit, so you are expected to be able to explain, change and discuss all parts of your submission with ease.

If you cannot make it to class on that date, please get in touch with the Professor so that alternative arrangements can be made. Nevertheless, you should still submit your work by the due date.